

Of course. Here is a detailed breakdown of the common and important methods for both `StringBuilder` and `StringBuffer`.

Fundamentally, `StringBuilder` and `StringBuffer` share an almost identical set of methods (API). The primary difference is that every modification method in `StringBuffer` is synchronized, making it thread-safe but slower. In contrast, `StringBuilder` methods are not synchronized, making them faster but suitable only for single-threaded use.

Below, the methods are listed with their signatures and explanations. You can apply them to both `StringBuilder` and `StringBuffer` objects.

Methods for `StringBuilder` and `StringBuffer`

1. Modification Methods (Changing the Content)

These methods alter the character sequence within the object. All these methods return the current `StringBuilder/StringBuffer` object to allow for method chaining.

Method	Description
<code>append(data)</code>	Appends the string representation of the data to the end. It's overloaded for all primitive types, <code>char[]</code> , <code>Object</code> , and <code>String</code> .
<code>insert(int offset, data)</code>	Inserts the string representation of the data at the specified offset. Also overloaded for all primitive types.
<code>delete(int start, int end)</code>	Removes the characters in a substring from the specified start index to the end-1 index.
<code>deleteCharAt(int index)</code>	Removes the character at the specified index.
<code>replace(int start, int end, String str)</code>	Replaces the characters from the start index to end-1 with the specified <code>String</code> .
<code>reverse()</code>	Reverses the character sequence.

setCharAt(int index, char ch)	Sets the character at the specified index to ch.
-------------------------------	--

Example of Chaining:

Java

```
public class ChainingDemo {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder("Java");  
  
        // Chaining methods together for concise code  
        sb.append(" is").insert(7, " Awesome").reverse();  
  
        // The operations happen in sequence:  
        // 1. "Java is"  
        // 2. "Java is Awesome"  
        // 3. "emosewA si avaJ"  
  
        System.out.println(sb);  
    }  
}
```

2. Querying and Status Methods (Getting Information)

These methods are used to get information about the state or content of the object.

Method	Description
length()	Returns the number of characters currently in the sequence.
charAt(int index)	Returns the char value at the specified index.
indexOf(String str)	Returns the index of the first occurrence of

	the specified substring. Returns -1 if not found.
<code>indexOf(String str, int fromIndex)</code>	Returns the index of the first occurrence of the substring, starting the search from <code>fromIndex</code> .
<code>lastIndexOf(String str)</code>	Returns the index of the last occurrence of the specified substring.
<code>substring(int start)</code>	Returns a new String containing the characters from start index to the end.
<code>substring(int start, int end)</code>	Returns a new String containing the characters from the start index to the end-1 index.
<code>toString()</code>	Returns a new String object representing the data in the <code>StringBuilder/StringBuffer</code> . This is the standard way to convert it back to an immutable string.

Example:

Java

```

public class QueryMethodsDemo {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Java Programming Language");

        System.out.println("Length: " + sb.length());
        System.out.println("Character at index 5: " + sb.charAt(5));
        System.out.println("Index of 'Lang': " + sb.indexOf("Lang"));

        // Extracting a part of the string
        String sub = sb.substring(5, 16); // from index 5 to 15
        System.out.println("Substring: " + sub); // "Programming"
    }
}

```

```
// Converting back to a String
String finalString = sb.toString();
System.out.println("Final String: " + finalString);
}
}
```

3. Capacity Management Methods

The `StringBuilder` and `StringBuffer` objects have a capacity, which is the amount of storage available for new characters. This capacity automatically grows as needed. However, you can manage it manually for performance tuning.

Method	Description
<code>capacity()</code>	Returns the current capacity (the number of characters it can hold before it must be resized). A new <code>StringBuilder</code> starts with a default capacity of 16.
<code>ensureCapacity(int minimumCapacity)</code>	Ensures that the capacity is at least equal to the given minimum. If the current capacity is less than the argument, a new internal array is allocated with a larger capacity.
<code>trimToSize()</code>	Attempts to reduce the storage used for the character sequence. It sets the capacity to be the same as the current length, freeing up any unused memory. Useful when you know no more appends will occur.
<code>setLength(int newLength)</code>	Sets the length of the character sequence. If <code>newLength</code> is less than the current length, the sequence is truncated. If it's greater, null characters (<code>\u0000</code>) are added to the end.

Example:

Java

```
public class CapacityDemo {  
    public static void main(String[] args) {  
        // Initial capacity is 16 + length of string = 20  
        StringBuilder sb = new StringBuilder("Four");  
  
        System.out.println("Content: " + sb);  
        System.out.println("Length: " + sb.length());    // 4  
        System.out.println("Capacity: " + sb.capacity()); // 20  
  
        // Ensure it can hold at least 50 characters  
        sb.ensureCapacity(50);  
        System.out.println("New Capacity: " + sb.capacity());  
  
        sb.append(" and seven years ago");  
        System.out.println("New Length: " + sb.length());  
  
        // Free up unused capacity  
        sb.trimToSize();  
        System.out.println("Trimmed Capacity: " + sb.capacity()); // Capacity will now equal length  
    }  
}
```

Summary Table: StringBuilder vs. StringBuffer

Feature	StringBuilder	StringBuffer
Introduced	Java 5	Java 1.0
Thread-Safety	Not thread-safe (unsynchronized methods)	Thread-safe (synchronized methods)
Performance	Faster due to lack of synchronization.	Slower due to synchronization overhead.

API	Virtually identical to StringBuffer.	Virtually identical to StringBuilder.
When to Use	In single-threaded environments (most cases).	In multi-threaded environments where a shared, mutable string is required.